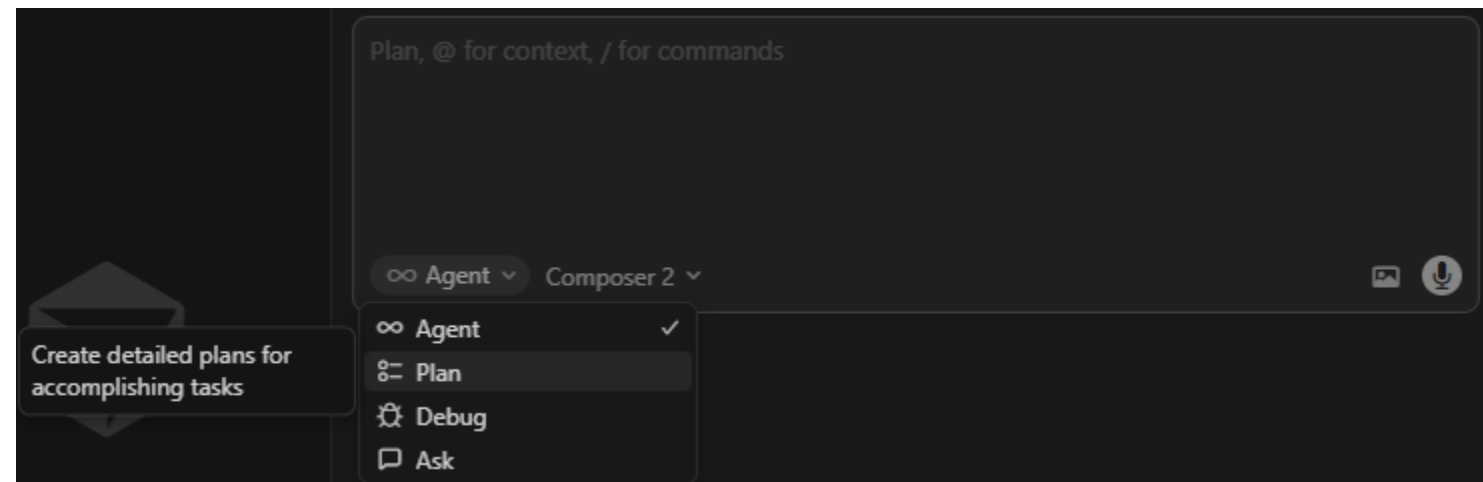


Second Exercise: AI-Assisted Development

- We tried vibe coding. Now let's take back control.
- In this exercise, we will be using Cursor to help us code again, but the project will be larger and we will try three different techniques: **Planning, Incremental Improvement, and Managing**
- The project these slides are working on is: *building an interactive call graph visualizer for x86 binaries*
- You can follow along with that goal, or create your own if you wish

Second Exercise: AI-Assisted Development

- Technique 1: Planning
 - Planning helps the agent do more complicated tasks, with a more well-specified goal for each step. It also allows you to see what design decisions the AI is making and approve or correct them as needed.
 - In the bottom left of the chat window, you can change your chat session type to “Agent” or “Plan” or “Ask”



Second Exercise: AI-Assisted Development

- Technique 1: Planning
 - **Agent:** this mode is the default that we used earlier, it can do basic coding tasks.
 - **Ask:** this mode is for asking questions when you DON'T want the AI to write code. You can ask about the code base, ask it for ideas, etc.
 - **Plan:** this mode is for larger, more complicated coding tasks. Before writing any code, this mode will create a plan document that is essentially a series of prompts that the agent will use to break down and accomplish larger coding tasks.

Second Exercise: AI-Assisted Development

- Technique 1: Planning
 - *(Optional)* You can also try co-designing your software's MVP with a different AI, like through ChatGPT. Almost any AI can be helpful to help synthesize a design document and help find holes and problems and solutions.
 - **MVP: Minimum Viable Product.** What is the smallest subset of your intended program that you can build to have something that runs and can be tested/used?

Second Exercise: AI-Assisted Development

ChatGPT ▾

 **High-Level Goal**

Build a **call graph visualizer** for x86 binaries that:

1. Parses binaries (ELF/PE)
2. Disassembles code
3. Identifies functions
4. Detects call relationships
5. Outputs an interactive or static graph

 **System Architecture**

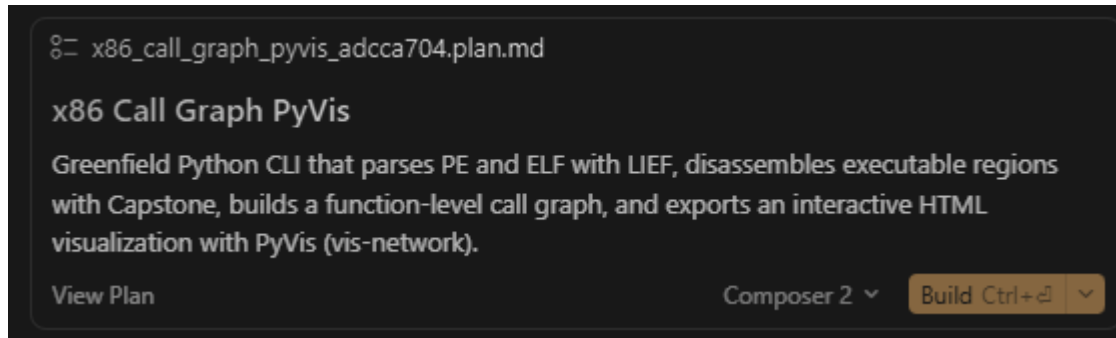
Core Pipeline

```
Binary Input
↓
Loader (ELF/PE parsing)
↓
Disassembler
↓
Function Analyzer
↓
Call Graph Builder
↓
Graph Exporter / Visualizer
```



Second Exercise: AI-Assisted Development

- Technique 1: Planning
 - After submitting a prompt in planning mode, you can view and edit the plan document
 - When ready, hit “Build”

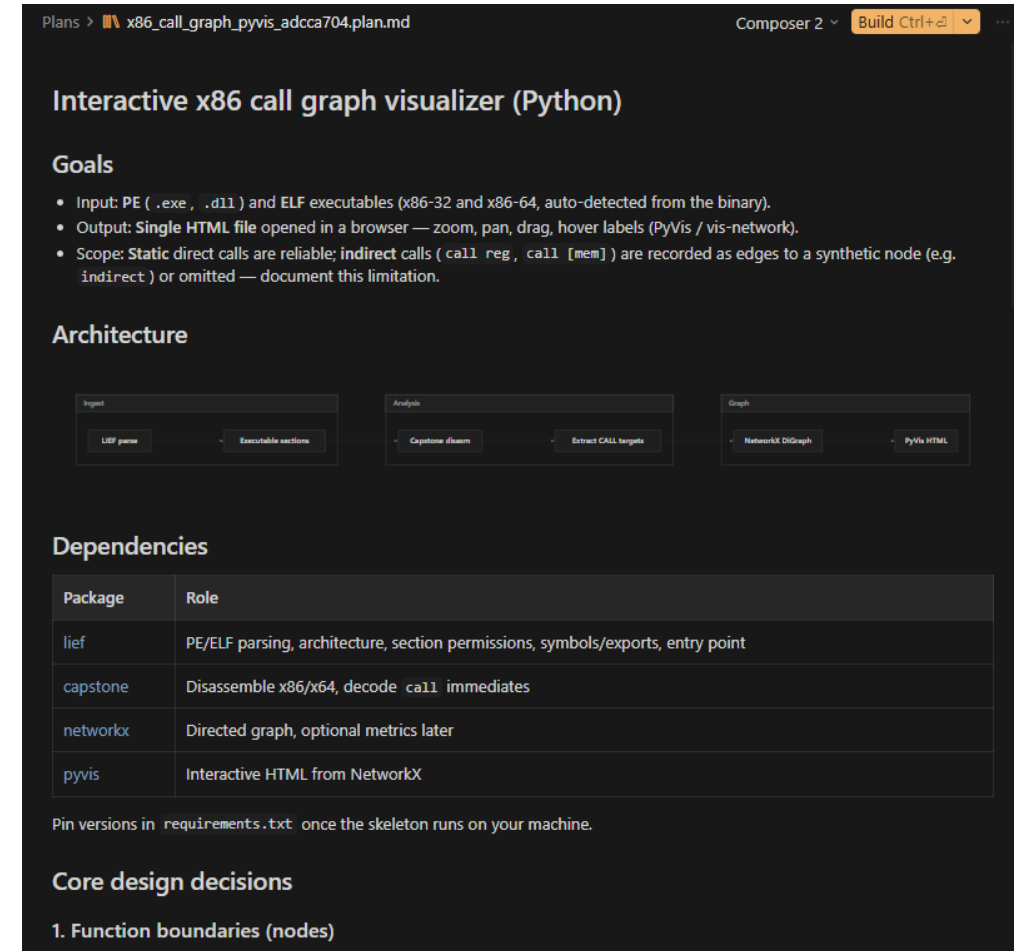


✎ x86_call_graph_pyvis_adcca704.plan.md

x86 Call Graph PyVis

Greenfield Python CLI that parses PE and ELF with LIEF, disassembles executable regions with Capstone, builds a function-level call graph, and exports an interactive HTML visualization with PyVis (vis-network).

View Plan Composer 2 Build Ctrl+⌘



Plans > x86_call_graph_pyvis_adcca704.plan.md Composer 2 Build Ctrl+⌘

Interactive x86 call graph visualizer (Python)

Goals

- Input: PE (.exe , .dll) and ELF executables (x86-32 and x86-64, auto-detected from the binary).
- Output: Single HTML file opened in a browser — zoom, pan, drag, hover labels (PyVis / vis-network).
- Scope: Static direct calls are reliable; indirect calls (call reg , call [mem]) are recorded as edges to a synthetic node (e.g. indirect) or omitted — document this limitation.

Architecture

```
graph LR
    subgraph Input
        LIEF_parser[LIEF parser] --> Executable_sections[Executable sections]
    end
    subgraph Analysis
        Capstone_disasm[Capstone disasm] --> Extract_CALL_targets[Extract CALL targets]
    end
    subgraph Graph
        NetworkX_DiGraph[NetworkX DiGraph] --> PyVis_HTML[PyVis HTML]
    end
```

Dependencies

Package	Role
lief	PE/ELF parsing, architecture, section permissions, symbols/exports, entry point
capstone	Disassemble x86/x64, decode call immediates
networkx	Directed graph, optional metrics later
pyvis	Interactive HTML from NetworkX

Pin versions in requirements.txt once the skeleton runs on your machine.

Core design decisions

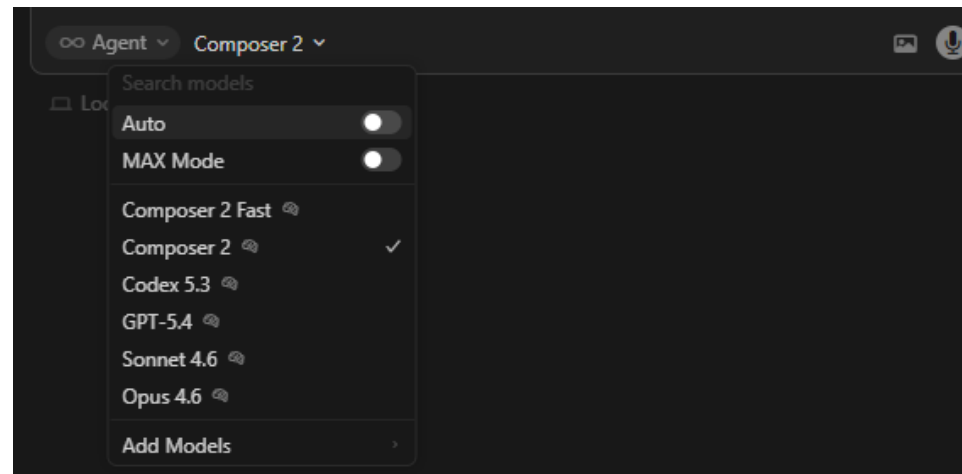
1. Function boundaries (nodes)

Second Exercise: AI-Assisted Development

- Technique 2: Incremental Improvement
 - Make incremental changes as small as possible, rather than trying to do the whole thing at once. The smaller and more defined the change – the more likely AI is to succeed.
 - **Too Big:** "I want the visualizer to be able to zoom in and out and move nodes around."
 - **OK:** "Add the ability to listen to keyboard input interactively."
 - **OK:** "When I press z, the visualization should zoom out by 25%."
 - **OK:** "Add the ability to listen to mouse input interactively."
 - **OK:** "When I click and drag a node, it should move (along with all the edges connected to it)"

Second Exercise: AI-Assisted Development

- Technique 2: Incremental Improvement
 - You can set the specific model used here
 - The model that gets used and its level of effort is important for determining how good the results you get are
 - Small, well-specified tasks can be safely given to stupider (cheaper) models
 - The big important stuff, like coming up with the plan, can benefit greatly from using a better model



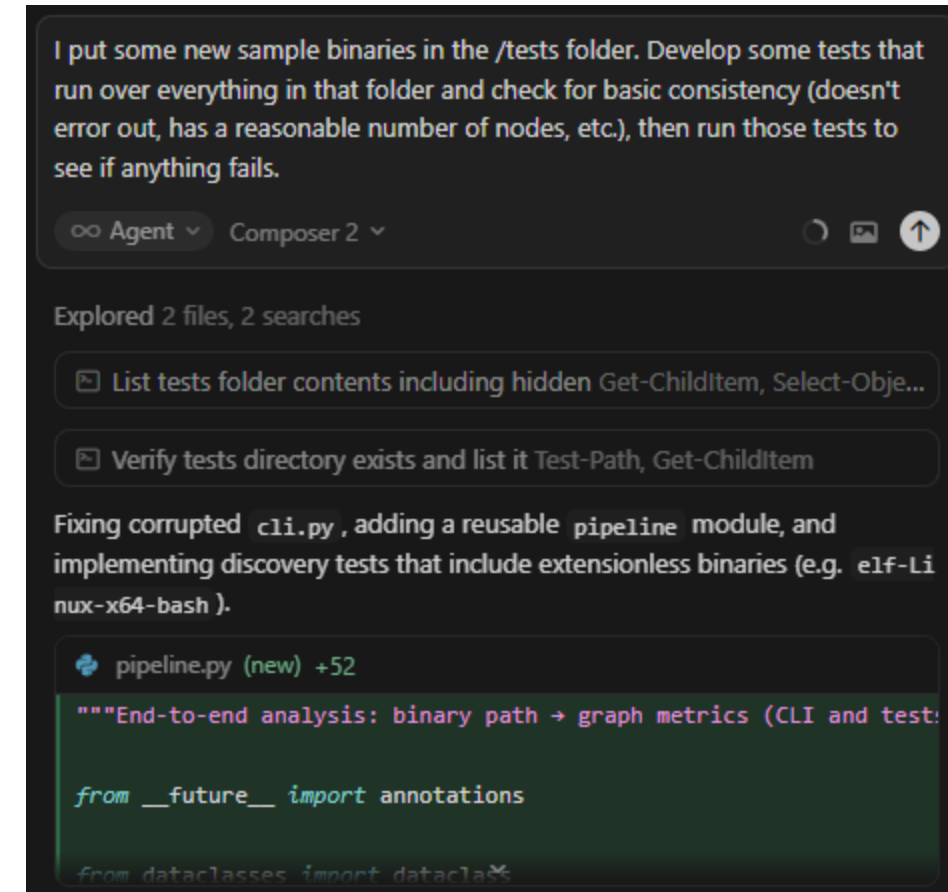
Second Exercise: AI-Assisted Development

- Technique 2: Incremental Improvement
 - Cursor has an autocomplete feature
 - Useful when you know exactly how to do something, it is a bit tricky, and you want it done right: make the surgical changes yourself
 - Try writing a comment to describe what you are about to do

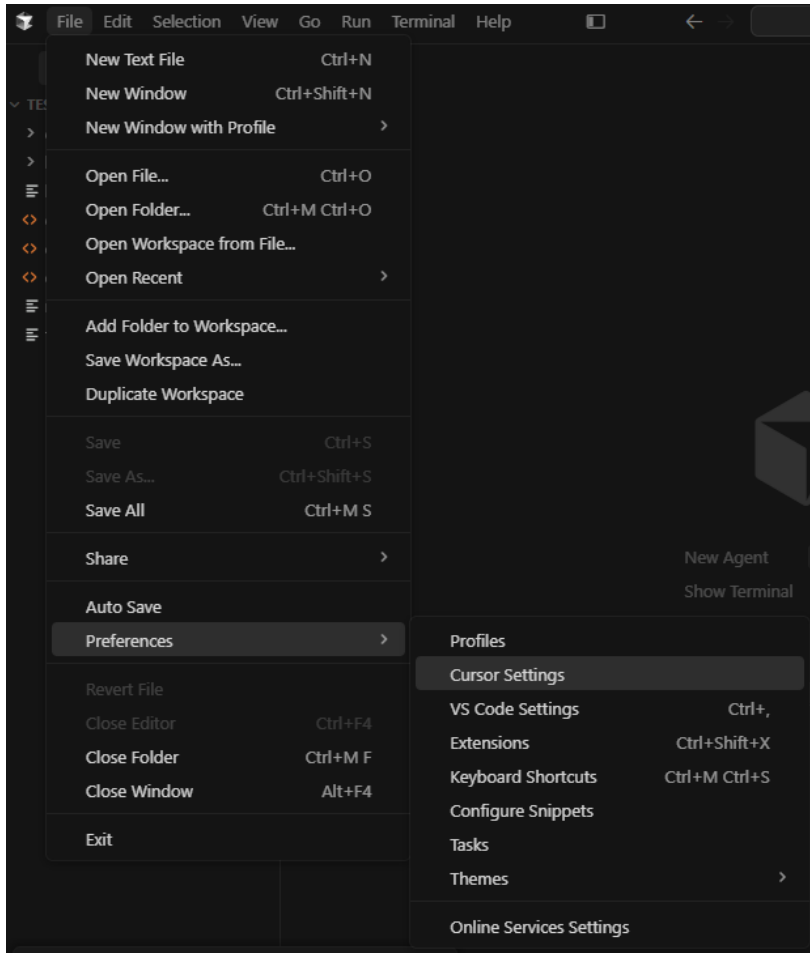
```
52     try:
53         binary, info = load_binary(args.input)
54     except ValueError as e:
55         print(f"error: {e}", file=sys.stderr)
56         return 1
57     entries = collect_function_entries(binary, info)
58     edges = extract_call_edges(
59         binary, info, entries, indirect_mode=args.indirect
60     )
61     g = build_call_graph(entries, edges)
62
63     // iterating over all the nodes in the graph and printing the name of the node and the address
    for node in g.nodes():
        print(f"Node: {node}, Address: {node.address}")
```

Second Exercise: AI-Assisted Development

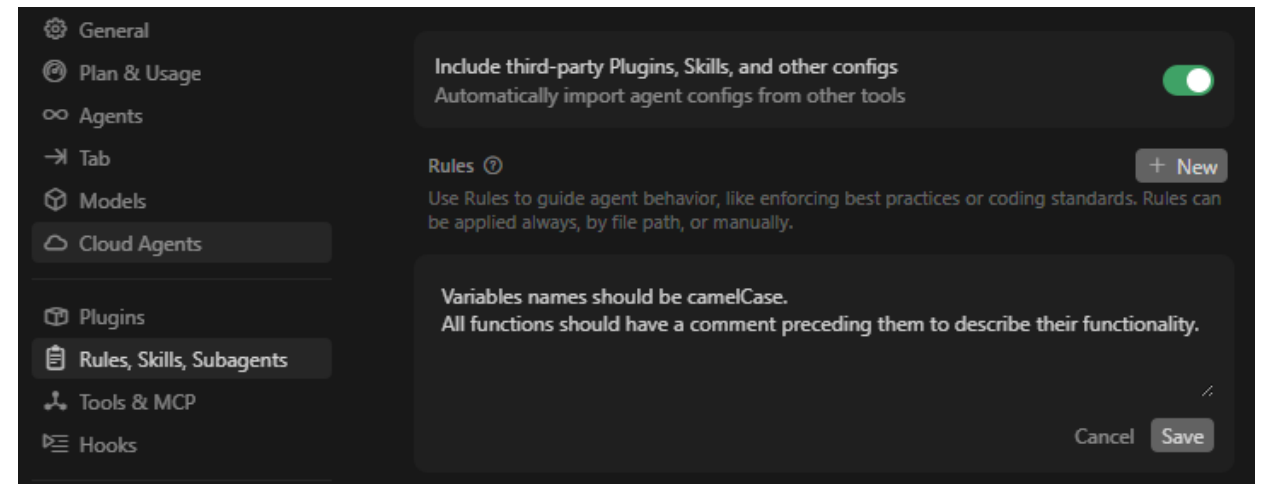
- Technique 3: Managing
 - The key to good AI coding practices is adopting many of the same good SWE practices that work for managing large software projects
 - Testing is an important one
 - If done right and well-specified, the AI can successfully debug problems in a loop on its own.
 - Be careful: in my experience, the AI decides removing tests is sometimes a good way to get things to pass the tests.



Second Exercise: AI-Assisted Development

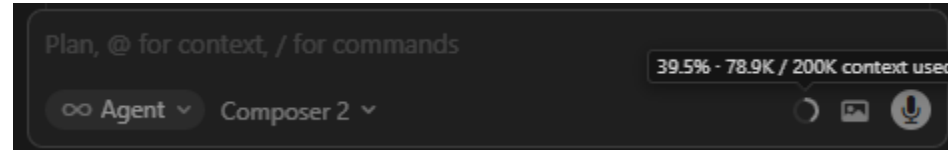


- Technique 3: Managing
 - Cursor has “rules” that allow you to more directly manage the system prompt provided to the AI. See: <https://cursor.com/docs/rules>
 - These rules are useful for project-level directives like formatting or coding style or overall architecture or “make sure you do/don’t do X”

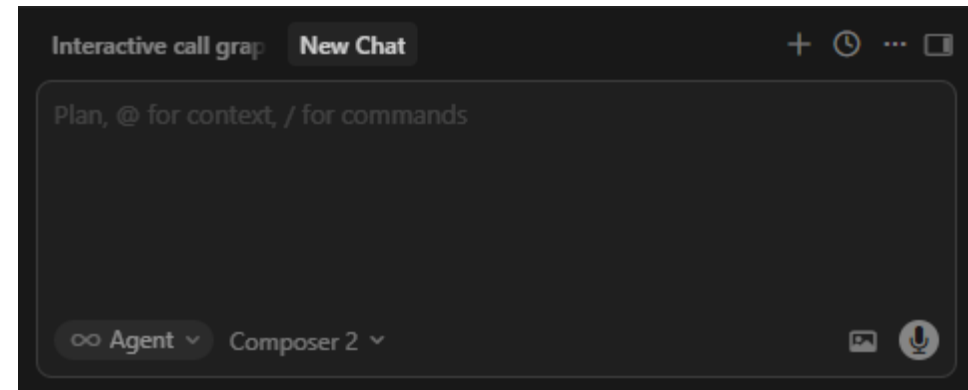


Second Exercise: AI-Assisted Development

- Technique 3: Managing
 - You can also more explicitly manage the context window of the AI.
 - See current context usage →



- Clear context in new chat window →



- Add specific context (file, URL, etc.) with @ →

